

CSI243 Functional Programming

Lab 6 Polymorphism and Overloading

Objectives

1. **Polymorphism**
2. **Overloading**

Background / Scenario

The word **polymorphism** literally means **having many shapes or forms**. A function is polymorphic if it **has many types**. That is, a polymorphic function can **act on more than one type**. A polymorphic function will have the **same definition at all these types**; a single definition will work over all types. This is a form of **generic/parametric polymorphism**. The type variable can take any value. Polymorphism allows us to write generic functions for different input types.

Overloading on the other hand is a form of **ad hoc polymorphism**. The symbol or function can be applied to different types, but the **function will have different definitions over different types**. The choice of the function is determined by the **context**, or **type class**. Type classes are used to provide a structured way to control overloading. A type class is a collection of types over which a function is defined. Haskell has several type classes including `Eq`, `Ord`, `Enum`, `Num`, `Integral`, `Show`, `Read`, `Fractional`. Members of a type class are called **instances**. A type is made into an instance by giving an implementation of the signature of that type

As usual, use the information to work on the lab.

For each of the functions, there should be five parts:

- i. Specify the signature of the function, that is, the arguments and the return type of the function
- ii. Write specifications for the function
- iii. Dry run the function
- iv. Implement using Haskell
- v. Test the function
- vi. The functions should be polymorphic

1. Define a function `lastElement` that returns the last element of a list.
2. Create a polymorphic function `revList` that reverses the elements of a list. That is `revList [1,2,3,4]` should return `[4,3,2,1]`.
3. Given a tuple, create a function `swapElements` that swaps the order of the elements in the tuple. That is `swapElements (4,9)` should return `(9,4)`.
4. Given two lists, the function `twoLists` should create a list of tuples using the elements of the two lists. For example,
`twoLists [1,2,3] [4,5,6]` should return `[(1,4), (2,5), (3,6)]`

=====

SUBMIT BEFORE THE END OF THE LAB ON THE LINK ON MOODLE